JavaScript / TypeScript

Task	Command	Notes
Version manager	`nvm`	`nvm install --lts` for latest LTS
New project	`npm init`	`npm create vite@latest` for frontend
Add dependency	`npm install` *pkg*	`--save-dev` for dev dependency
Install deps	`npm install`	installs from `package-lock.json`
Build	`tsc`	or `npm run build`
Run	`node script.js`	`npx tsx script.ts` for TypeScript
REPL	`node`	
Test	`vitest`	or `jest`; `npx vitest`
Format	`prettier --write .`	`--check` for dry run
Lint	`eslint .`	`--fix` to autofix
LSP	`typescript-language-server`	started by nvim and Emacs eglot

Variables & Types

`const x = v`	Immutable binding
`let x = v`	Mutable binding
`x: number = 0`	TS type annotation
`x?: string`	Optional property
`typeof x`	Runtime type check
`x as T`	TS type assertion
`x satisfies T`	TS type check (4.9+)

Functions

`(x: T): R => expr`	Arrow function
`function f(x: T): R`	Named function
`(...args: T[])`	Rest params
`f(a, ...xs)`	Spread args
`{x = default}`	Default params
`function* f()`	Generator
`yield v`	Yield value

Destructuring & Spread

`const {a, b} = obj`	Object destructure
`const [x, y] = arr`	Array destructure
`const {a: renamed} = o`	Rename
`const {a, ...rest} = o`	Rest in destructure
`{...obj, key: v}`	Object spread
`[...arr, x]`	Array spread

Common Patterns

`x ?? fallback`	Nullish coalescing
`x?.prop`	Optional chaining
`x ??= v`	Nullish assign
`cond ? a : b`	Ternary
`` `${expr}` ``	Template literal
`Symbol("desc")`	Unique symbol
`for (const x of xs)`	Iterate values
`for (const k in obj)`	Iterate keys

Array Methods

`.map(f)`	Transform each
`.filter(p)`	Keep matching
`.reduce(f, init)`	Fold
`.find(p)`	First match
`.findIndex(p)`	First match index
`.some(p) / .every(p)`	Any / all
`.flat() / .flatMap(f)`	Flatten
`.includes(v)`	Contains
`.sort((a,b) => a - b)`	Sort
`.slice(i, j)`	Subarray (copy)
`.splice(i, n, ...items)`	Mutate in-place
`Array.from(iterable)`	From iterable
`[...new Set(xs)]`	Unique values
`.at(-1)`	Last element

Object Methods

`Object.keys(o)`	Keys array
`Object.values(o)`	Values array
`Object.entries(o)`	[key, value] pairs
`Object.fromEntries(es)`	Pairs to object
`Object.assign(t, s)`	Merge / copy
`structuredClone(o)`	Deep clone

Async / Await

`async function f()`	Async function
`await promise`	Await result
`Promise.all(ps)`	Wait for all
`Promise.allSettled(ps)`	All results
`Promise.race(ps)`	First settled
`Promise.any(ps)`	First fulfilled
`new Promise((res, rej) =>)`	Create promise
`.then(f).catch(e)`	Chain / handle

Error Handling

`try { } catch (e) { }`	Try/catch
`finally { }`	Always runs
`throw new Error("msg")`	Throw error
`class E extends Error`	Custom error
`e instanceof TypeError`	Error type check

Modules

`export const x = v`	Named export
`export default f`	Default export
`import {x} from "m"`	Named import
`import x from "m"`	Default import
`import * as M from "m"`	Namespace import
`import type {T} from`	TS type-only import

Classes

`class C extends B`	Class declaration
`constructor(x: T)`	Constructor
`private / #field`	Private field
`readonly prop: T`	TS readonly
`get x() / set x(v)`	Getter / setter
`static method()`	Static method
`implements I`	TS interface impl

TypeScript — Type System & Node.js

Basic Types

`string, number, boolean`	Primitives
`null, undefined`	Nullish types
`unknown`	Safe any
`never`	Unreachable
`void`	No return
`any`	Escape hatch
`bigint, symbol`	Other primitives

Interfaces & Type Aliases

`interface I { x: T }`	Interface
`type T = A | B`	Union type
`type T = A & B`	Intersection type
`type T = {[k: string]: V}`	Index signature
`extends`	Interface extension
`interface I { (x: T): R }`	Callable

Generics

`function f<T>(x: T): T`	Generic function
`<T extends Constraint>`	Bounded generic
`<T = Default>`	Default type
`interface I<T>`	Generic interface
`type Box<T> = {v: T}`	Generic alias

Utility Types

`Partial<T>`	All optional
`Required<T>`	All required
`Readonly<T>`	All readonly
`Record<K, V>`	Key-value map
`Pick<T, K>`	Subset of keys
`Omit<T, K>`	Exclude keys
`Exclude<U, T>`	Remove from union
`Extract<U, T>`	Keep in union
`NonNullable<T>`	Remove null/undef
`ReturnType<F>`	Function return type
`Parameters<F>`	Function params tuple
`Awaited<T>`	Unwrap Promise

Advanced Types

`T extends U ? A : B`	Conditional type
`infer R`	Infer in conditional
`{[K in keyof T]: V}`	Mapped type
`as`	Key remapping
`keyof T`	Keys as union
`T[K]`	Indexed access
`template literal types`	`` `${A}-${B}` ``
`const assertions`	`as const`

Type Guards

`typeof x === "string"`	Primitive guard
`x instanceof C`	Class guard
`"key" in obj`	Property guard
`x is T`	Custom type pred
`asserts x is T`	Assertion function

Discriminated Unions

`type R = A | B`	Union type
`{kind: "a", ...}`	Tag field
`switch (r.kind)`	Narrow by tag
`exhaustive check`	`never` in default

Node.js — File System

`import fs from "node:fs/promises"`	Async FS
`fs.readFile(f, "utf8")`	Read file
`fs.writeFile(f, data)`	Write file
`fs.readdir(dir)`	List directory
`fs.stat(f)`	File info
`fs.mkdir(d, {recursive})`	Create dirs
`fs.rm(f)`	Remove file

Node.js — Path & Process

`import path from "node:path"`	Path utils
`path.join(a, b)`	Join paths
`path.resolve(p)`	Absolute path
`path.basename(p)`	Filename
`path.extname(p)`	Extension
`process.argv`	CLI arguments
`process.env.KEY`	Environment var
`process.cwd()`	Working directory
`process.exit(code)`	Exit

Node.js — HTTP & Child Process

`fetch(url, opts)`	HTTP request
`await res.json()`	Parse JSON
`new URL(s)`	Parse URL
`URLSearchParams`	Query params
`exec(cmd, cb)`	Run shell cmd
`spawn(cmd, args)`	Spawn process

`from "node:child_process"`

Testing (Vitest)

`describe("...", () => {})`	Test suite
`it("...", () => {})`	Test case
`expect(x).toBe(y)`	Strict equal
`expect(x).toEqual(y)` Deep equal	
`expect(f).toThrow()` Throws	
`vi.fn()`	Mock function
`vi.spyOn(obj, "method")`	Spy on method
`beforeEach / afterEach`	Setup / teardown

Package Config

`"type": "module"`	ESM in package.json
`"strict": true`	TS strict mode
`tsconfig.json`	TS configuration
`.eslintrc / eslint.config`	ESLint config
`.prettierrc`	Prettier config